# Horizontal Scaling Architecture Specification

## For a Reactive Database Engine Built on Embedded Storage

---

## Executive Summary

This document specifies the horizontal scaling architecture for a single-binary reactive database engine written in Rust. The core insight is that the embedded storage engine (redb) operates within a single process and does not scale horizontally itself — instead, scaling is achieved by distributing isolated tenant databases across multiple nodes, with a coordination layer for routing and a commit log for reactive query invalidation and edge replication.

The architecture combines three proven patterns — database-per-tenant, log-driven reactive invalidation, and embedded edge replicas — into a five-phase implementation plan that delivers horizontal scaling without distributed transactions, without consensus on the hot path, and without sacrificing the sub-millisecond subscription update latency that defines the product.

---

## 1. Why Redb Does Not Scale Horizontally (And Why That's Fine)

Redb is a pure-Rust embedded key-value store using copy-on-write B-trees. It operates within a single process, reads and writes to a single file, and provides ACID transactions with serializable isolation through MVCC. It is architecturally identical to SQLite and LMDB in this regard: it is a storage engine, not a distributed database.

Redb has no concept of sharding, partitioning, replication, or multi-node coordination. It cannot split its data across machines, and it cannot serve concurrent writers from multiple processes. These are not bugs — they are fundamental design choices that make redb fast, correct, and simple. Attempting to bolt distribution onto an embedded engine would compromise all three.

The correct approach is to treat redb as the storage primitive inside a larger system that handles distribution at a higher layer. The question is not "how does redb scale?" but "how do we distribute the workload so each redb instance stays small, fast, and local?"

---

## 2. The Five Horizontal Scaling Patterns

### 2.1 Pattern A: Shared-Nothing Range Sharding

**Used by:** CockroachDB, TiDB, YugabyteDB, Google Spanner

**How it works:** The entire keyspace is divided into contiguous ranges (typically 64-512MB each). Each range is replicated across multiple nodes using consensus (Raft or Paxos). A routing layer maps keys to ranges and ranges to nodes. As data grows, ranges split automatically. As load shifts, ranges migrate between nodes.

**Architecture:**

```
Client → Router → Range Leader (Node A) → Raft → Followers (Node B, C)
                                        → Embedded Storage (Pebble/RocksDB)
```

**Strengths:**

- True horizontal write scaling — add nodes, add throughput
- Automatic rebalancing under load
- Strong consistency via consensus per range
- Transparent to the application — looks like a single database

**Weaknesses:**

- Enormous implementation complexity (CockroachDB is >1M lines of Go)
- Cross-range transactions require distributed 2PC, adding latency and failure modes
- Reactive query invalidation across ranges requires coordination — a subscription that touches data in Range A and Range B needs both ranges to participate in invalidation, adding network hops to every subscription update
- Range splits under load create transient performance degradation
- Consensus on the write path adds 1-2 network round-trips of latency to every write

**Fit for reactive database:** Poor. The distributed coordination on every write directly conflicts with the sub-millisecond subscription update target. Cross-range subscriptions are architecturally expensive. This pattern solves a different problem (scaling a single logical database to petabyte scale) than what most reactive app backends need.

---

### 2.2 Pattern B: Disaggregated Storage and Compute

**Used by:** Amazon Aurora, Neon, Google AlloyDB, Microsoft Socrates, Snowflake

**How it works:** The database is split into a stateless compute layer (query processing, transaction management) and a shared durable storage layer (page/block storage with replication). The compute node sends WAL entries to the storage layer, which handles replication and

materialization. Multiple read-only compute nodes can be spun up to serve read traffic, all reading from the same shared storage.

**Architecture:**

```
Write Path:  Client → Primary Compute → WAL → Storage Layer (3-way replicated)
Read Path:   Client → Read Replica Compute → Page Request → Storage Layer
                                                → Local File Cache (SSD)
```

**Strengths:**

- Elegant separation of concerns — compute and storage scale independently
- Read scaling is trivial (add read replicas that pull from shared storage)
- Scale-to-zero compute reduces cost for idle workloads
- Storage is "bottomless" — backed by object storage (S3)
- Copy-on-write branching enables instant database snapshots (Neon achieves this in <1 second regardless of database size)
- Compute nodes are stateless, enabling fast failover

**Weaknesses:**

- Single writer — the primary compute node is a bottleneck for writes
- Network hop between compute and storage adds latency on buffer pool misses (mitigated by local SSD cache, but still present)
- Subscription routing is complex — the primary knows about writes, but read replicas serve subscriptions, so invalidation signals must flow from primary to replicas
- Requires a sophisticated storage layer (Neon's is written in Rust, ~100k lines)
- Not a fit for the "single binary" deployment target — inherently requires multiple processes

**Fit for reactive database:** Moderate. Good for read-heavy reactive workloads where subscriptions can be served by read replicas. But the multi-process architecture conflicts with the single-binary goal, and the network hop between compute and storage adds tail latency to subscription updates. Best used as an escape hatch for very large tenants, not as the default architecture.

---

### 2.3 Pattern C: Database-per-Tenant

**Used by:** Turso, PlanetScale (Vitess), many SaaS applications

**How it works:** Each tenant (user, organization, workspace) gets their own isolated database instance. A routing layer maps tenant identifiers to database instances. Databases are distributed across nodes — each node hosts many tenant databases. Scaling means adding nodes and distributing tenants across them.

**Architecture:**

```
Client → Tenant Router → Node A → Tenant 1 Database (redb file)
                                → Tenant 2 Database (redb file)
                                → Tenant 3 Database (redb file)
                       → Node B → Tenant 4 Database (redb file)
                                → Tenant 5 Database (redb file)
                                → ...
```

**Strengths:**

- Dead simple to reason about — no distributed transactions, no cross-tenant coordination
- Perfect data isolation — each tenant's data is a separate file, enabling per-tenant encryption, backup, restore, export, and deletion
- Reactive queries are trivially scoped — a subscription only ever touches one tenant's data, so invalidation is purely local with zero network coordination
- Natural horizontal scaling — distribute tenants across nodes, each node runs the same binary
- Tenant migration is a file copy plus traffic redirect — no complex rebalancing
- Compliance-friendly — GDPR "right to deletion" is literally deleting a file
- Schema migrations can be rolling — upgrade tenants incrementally, not all at once
- Turso demonstrates this at scale: 10,000+ databases per node, millions across a fleet

**Weaknesses:**

- Cross-tenant queries are impossible within the database layer — requires application-level aggregation or a separate analytics pipeline
- A single very large tenant cannot be split — vertical scaling only for individual tenants
- Metadata management (tenant → node mapping) requires a coordination layer
- Schema changes must be applied to every tenant database (thousands or millions of them)
- Connection overhead if each tenant database is a separate file descriptor (mitigated by connection pooling and lazy opening)
- Monitoring and observability across thousands of databases requires purpose-built tooling

**Fit for reactive database:** Excellent. This is the natural scaling model for reactive app backends. The vast majority of SaaS applications have strong tenant boundaries — users in Organization A never see data from Organization B. Reactive subscriptions are tenant-scoped by definition. This pattern eliminates all distributed coordination from the subscription hot path.

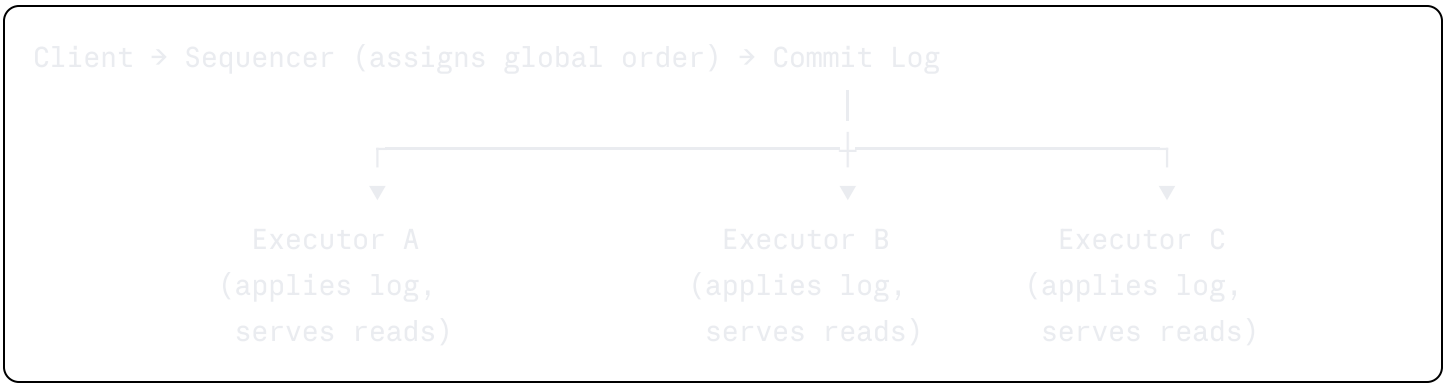
---

### 2.4 Pattern D: Deterministic Log with Unbundled Roles

**Used by:** FoundationDB, Calvin, FaunaDB (now Fauna)

**How it works:** All transactions are sequenced into a deterministic, ordered log before execution. Multiple executor nodes consume the log and apply transactions in the same order, producing identical state. Because execution is deterministic, any node that has processed the same prefix of the log is guaranteed to be in the same state. Reads go to any executor that is sufficiently caught up.

**Architecture:**




**Strengths:**

- Clean separation between ordering (sequencer) and execution (executors)
- Read scaling by adding executors
- Recovery is simple — replay the log from the last checkpoint
- The commit log is a perfect source for reactive query invalidation — scan the log to determine which subscriptions are affected by each committed transaction
- The commit log can be streamed to edge replicas for near-real-time replication
- No distributed locking or 2PC — the log provides total ordering

**Weaknesses:**

- The sequencer is a serialization bottleneck — all writes flow through a single ordering point
- Transaction latency includes the sequencer hop plus log replication
- Requires knowing the read/write set before execution (Calvin-style) or accepting abort/retry (FoundationDB-style)

- FoundationDB imposes a 5-second transaction time limit specifically because of this architecture
- More complex than database-per-tenant for the common case

**Fit for reactive database:** The log-driven invalidation idea is extremely valuable, even if the full unbundled architecture is not adopted. The commit log is the ideal data structure for deriving subscription updates — it tells you exactly what changed, in what order, enabling efficient incremental invalidation. This idea should be incorporated into the database-per-tenant architecture as a per-tenant commit log.

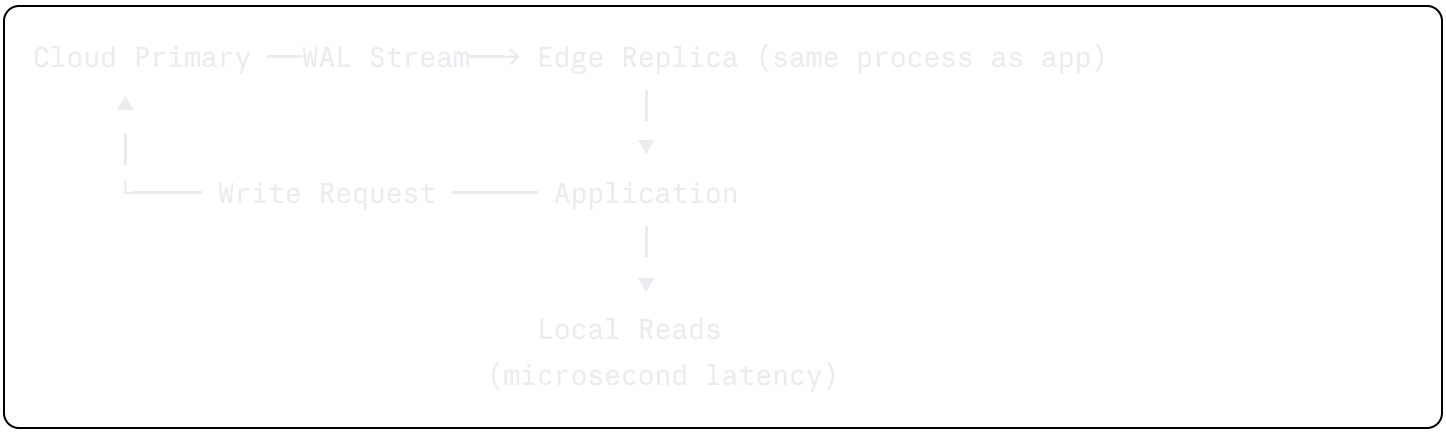
---

### 2.5 Pattern E: Edge Replication with Embedded Replicas

**Used by:** Turso (embedded replicas), Electric SQL, Replicache/Zero, Litestream

**How it works:** A primary database in the cloud handles all writes. Changes are streamed (via WAL shipping, CDC, or a custom sync protocol) to read-only replicas embedded in the application process — either at edge locations, on mobile devices, or in the browser (via WASM). Reads hit the local replica with zero network latency. Writes go to the cloud primary and propagate back.

**Architecture:**




**Strengths:**

- Reads are essentially free — microsecond latency from an in-process database
- Perfect for reactive queries — the replica can evaluate subscriptions locally without any network call
- Works offline — the embedded replica continues serving reads when disconnected
- Write-back when reconnected, with conflict resolution
- Global distribution without edge infrastructure — the replica is embedded in the client

- Turso demonstrates sub-millisecond read latency with embedded replicas syncing to a cloud primary

**Weaknesses:**

- Write latency includes the round-trip to the cloud primary
- Eventual consistency — there is a window after a write where the local replica has not yet received the update (typically milliseconds, but can be longer under load or poor network)
- If multi-primary writes are allowed (multiple clients writing offline), conflict resolution is required (CRDTs or application-defined merge)
- Storage budget on the client — mobile devices and browsers have limited space
- Partial replication is necessary (you can't sync the entire database to every client)

**Fit for reactive database:** Outstanding complement to database-per-tenant. The per-tenant commit log streams changes to embedded replicas where reactive queries evaluate locally. This eliminates the server from the read path entirely for subscribed data, achieving the ultimate reactive performance: local-first reads with server-authoritative writes.

---

## 3. Recommended Architecture: Patterns C + D + E Combined

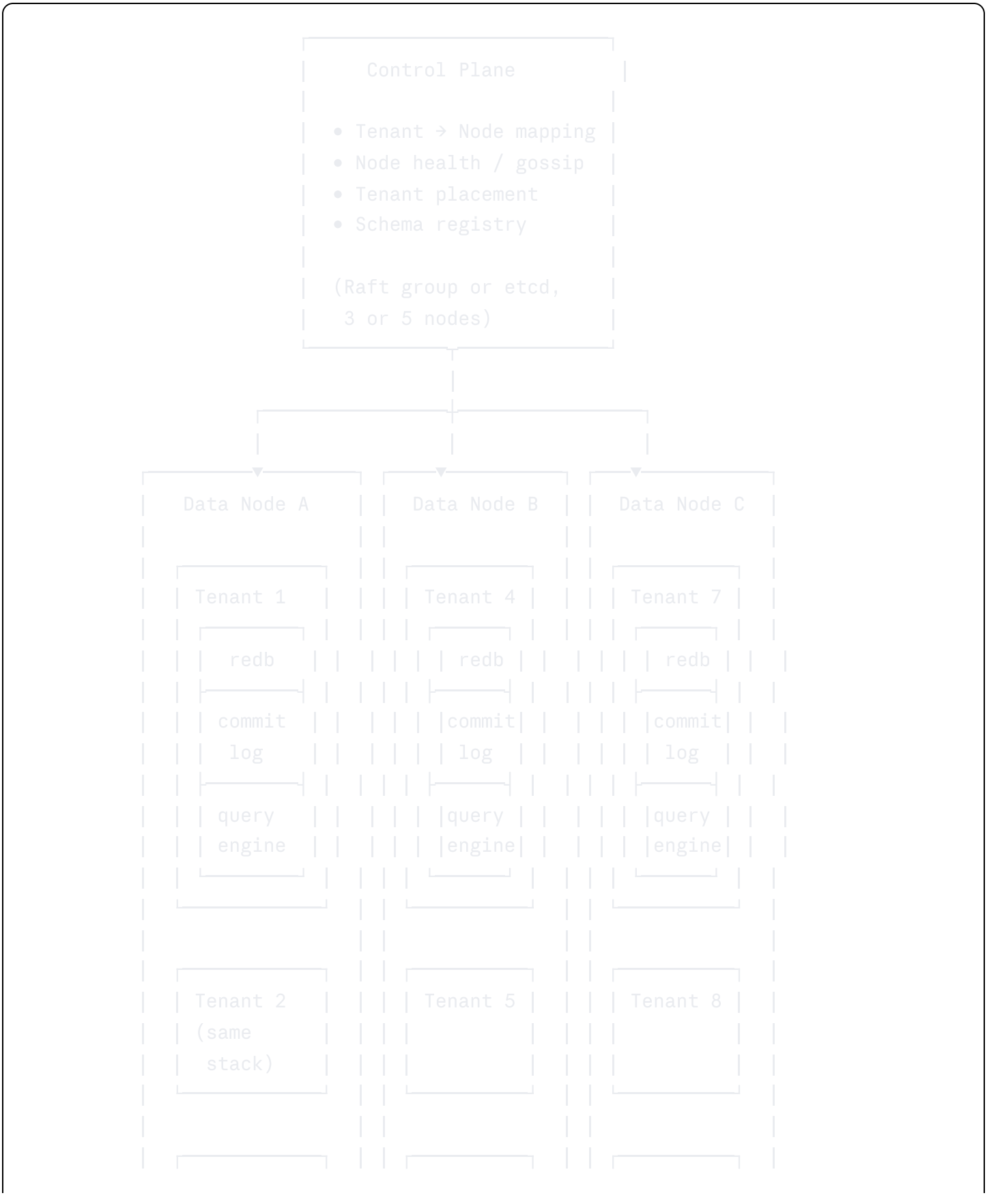
The optimal architecture for a reactive database engine combines database-per-tenant isolation (Pattern C) with per-tenant commit logs for reactive invalidation (Pattern D) and embedded edge replicas for local-first reactive queries (Pattern E).

This combination is chosen because:

1. **Database-per-tenant eliminates distributed transactions entirely.** No cross-tenant coordination on the write path. No 2PC. No distributed locking. Each tenant's transactions are serializable within their own database.
2. **Per-tenant commit logs provide the ideal reactive primitive.** Every committed write produces a log entry. Subscription invalidation is derived from scanning the log, not from tracking read sets during query execution. The log can be streamed to any number of consumers — edge replicas, analytics pipelines, audit systems.
3. **Embedded replicas achieve zero-latency reactive reads.** The client holds a local copy of the subscribed data. Reactive queries evaluate against the local replica. The server only needs to stream the commit log — it does not need to re-evaluate queries or push results.
4. **Horizontal scaling is node-level, not data-level.** You scale by adding nodes and distributing tenants, not by partitioning data within a tenant. This is operationally simple and avoids all the complexity of range-based sharding.

## 4. Detailed Architecture

### 4.1 System Topology



### 4.2 Control Plane

The control plane manages cluster-wide metadata. It does not sit on the hot path of any read or write operation. It handles:

**Tenant Placement:**

- Maintains a mapping of tenant_id → node_id
- Assigns new tenants to the least-loaded node (by tenant count, storage bytes, or subscription count)
- Handles tenant migration requests (move tenant X from Node A to Node B)

**Node Membership:**

- Tracks which data nodes are alive via heartbeat/gossip
- Detects node failures and triggers tenant reassignment
- Manages node join/leave for cluster scaling

**Schema Registry:**

- Stores the current schema version for each tenant (or globally, if all tenants share a schema)
- Coordinates rolling schema migrations across tenant databases

**Implementation Options:**

- For small clusters (3-5 nodes): embed a Raft group in the data nodes using `openraft` in Rust. One node is the Raft leader and handles placement decisions. This avoids an external dependency.

- For larger clusters (10+ nodes): use etcd or a FoundationDB-backed coordination service.
- For the single-node case: the control plane is a simple in-memory data structure. No consensus needed.

**Consistency requirement:** The tenant → node mapping must be **eventually consistent with a bias toward availability.** A stale mapping means a client connects to the wrong node, which can redirect (HTTP 307 or WebSocket redirect). This is safe because the tenant's actual data only exists on one node at a time.

### 4.3 Data Node

Each data node is a single process running the same binary. It hosts multiple tenant databases and handles all read/write traffic for those tenants.

**Components within a data node:**

#### 4.3.1 Tenant Manager

- Opens and closes tenant databases on demand (lazy initialization — don't open all 10,000 redb files at startup)
- Maintains an LRU cache of open tenant databases (keep the most active tenants open, close idle ones to conserve file descriptors and memory)
- Handles tenant creation (initialize a new redb file with the schema) and deletion (close and delete the file)
- Enforces per-tenant resource limits (max storage size, max concurrent connections, max subscriptions)

#### 4.3.2 Per-Tenant Database Stack

Each tenant database contains:

**Storage Engine (redb):**

- A single redb file containing all tables, indexes, and metadata for one tenant
- ACID transactions with serializable isolation via copy-on-write B-trees
- MVCC snapshots for consistent reads during query evaluation

**Commit Log:**

- An append-only log of all committed transactions for this tenant
- Each log entry contains: sequence number, timestamp, the write set (which keys were inserted/updated/deleted), and a checksum
- The log is the source of truth for reactive query invalidation and edge replication

- Persisted to disk (can be a separate file or a reserved table within redb)
- Truncated after a configurable retention period or after all replicas have confirmed receipt

**Reactive Query Engine:**

- Maintains a set of active subscriptions for this tenant
- Each subscription is a registered query plus a connection to push results to
- When a transaction commits, the engine scans the commit log entry's write set against the subscriptions' dependency sets
- Dependencies can be tracked at different granularities:
  - Table-level: "this subscription reads from the `messages` table" (coarsest, cheapest)
  - Index-range-level: "this subscription reads messages where channel_id = 42" (medium)
  - Document-level: "this subscription reads document with _id = abc123" (finest, most expensive)
- Start with table-level dependencies for v1, refine to index-range-level for v2
- When a dependency match is found, re-evaluate the query and push the new result to the client (full re-evaluation for v1, incremental delta for v2)

**WASM Plugin Runtime:**

- Executes user-defined functions (mutations, computed fields, triggers) in a sandboxed WASM environment
- Each function invocation is a transaction — reads and writes are tracked for both OCC conflict detection and subscription invalidation
- Uses `wasmtime` for execution with fuel-based resource limiting (prevent infinite loops, bound memory usage)

#### 4.3.3 Connection Manager

- HTTP/2 server for one-shot requests (mutations, action invocations, admin API)
- WebSocket server for long-lived subscriptions
- Each WebSocket connection is associated with one tenant and zero or more subscriptions
- Multiplexes subscription updates over a single WebSocket connection per client
- Connection authentication via JWT — the tenant_id is extracted from the token
- If a client connects to the wrong node (stale tenant mapping), the node responds with a redirect to the correct node

#### 4.3.4 Log Streamer

- Streams the per-tenant commit log to external consumers:
  - Edge replicas (for local-first reads)
  - Analytics pipelines (for dashboards and reporting)
  - Audit systems (for compliance)
  - Backup services (for disaster recovery)
- Uses a pull-based protocol — consumers track their own cursor position in the log and request the next batch of entries
- Supports backpressure — if a consumer falls behind, it can catch up by reading from the persisted log

### 4.4 Tenant Routing

When a client connects, the system must route the request to the correct data node.

**Option A: DNS-based routing**

- Each tenant gets a subdomain: `tenant-abc.db.example.com`
- DNS resolves to the node hosting that tenant
- Simple but slow to update (DNS TTL) and limited by DNS record limits

**Option B: Application-layer routing with a thin proxy**

- A stateless proxy (or load balancer) receives all connections
- Extracts the tenant_id from the JWT or URL path
- Looks up the tenant → node mapping (cached locally, refreshed periodically from the control plane)
- Proxies the connection to the correct data node
- Can be implemented in the same binary (the proxy is just another mode of the same executable)

**Option C: Client-side routing**

- The client SDK fetches the tenant → node mapping on initialization
- Connects directly to the correct data node
- The mapping is cached and refreshed on redirect or periodically
- Lowest latency (no proxy hop) but requires SDK cooperation

**Recommendation:** Start with Option B (application-layer proxy) for simplicity. The proxy is stateless and can run on every node (each node acts as both a proxy for incoming requests and a data node for its local tenants). Migrate to Option C (client-side routing) for production performance.

### 4.5 Tenant Migration

Moving a tenant from Node A to Node B:

1. **Freeze writes** on the tenant database (queue incoming mutations, continue serving reads)
2. **Flush the commit log** to ensure all committed data is on disk
3. **Copy the redb file and commit log** from Node A to Node B (this is a file copy — can use rsync, SCP, or object storage as an intermediary)
4. **Open the database on Node B** and verify integrity (checksum validation)
5. **Update the tenant → node mapping** in the control plane (atomic compare-and-swap)
6. **Redirect traffic** — Node A starts returning redirects for this tenant. Clients reconnect to Node B.
7. **Drain queued mutations** — replay any mutations that were queued during the freeze on Node B
8. **Delete the tenant data from Node A** after confirming Node B is serving successfully

**Expected migration time for a 1GB tenant database:** 5-30 seconds depending on network speed, with <1 second of write unavailability (during the freeze and redirect).

For zero-downtime migration, a dual-write approach can be used: both nodes accept writes during migration, with the commit log used to reconcile any writes that landed on the old node after the copy was taken. This adds complexity but eliminates the write freeze.

### 4.6 Tenant Durability and Backup

Each tenant's data is a self-contained set of files (redb database + commit log). Backup strategies:

**Continuous backup via commit log shipping:**

- Stream the commit log to object storage (S3, GCS, R2) in near-real-time
- A full base backup (copy of the redb file) is taken periodically (daily or weekly)
- Point-in-time recovery is achieved by restoring the base backup and replaying the commit log to the desired timestamp
- This is identical to Postgres's WAL archiving model, but per-tenant

**Snapshot backup:**

- Since redb uses copy-on-write B-trees, a consistent snapshot can be taken without pausing writes — just copy the file at a point where the root pointer is consistent
- Alternatively, use filesystem-level snapshots (ZFS, Btrfs) or LVM snapshots

**Replication for durability:**

- For tenants requiring high durability, replicate the commit log synchronously to a second node before acknowledging writes
- This turns the per-tenant commit log into a replicated log (Raft for a single tenant) without requiring the full shared-nothing sharding complexity
- Only needed for premium/enterprise tenants — most tenants can rely on periodic backup to object storage

---

## 5. Implementation Phases

**Phase 1: Single Node, Multi-Tenant (Weeks 1-8)**

**Goal:** A single binary that hosts thousands of tenant databases with reactive subscriptions.

**Deliverables:**

- Tenant manager with lazy database opening and LRU eviction
- Per-tenant redb database with schema enforcement
- Per-tenant commit log (append-only file)
- Reactive query engine with table-level dependency tracking
- WebSocket subscription multiplexing
- HTTP API for mutations and admin operations
- JWT-based authentication with tenant_id extraction
- CLI for tenant creation, deletion, and listing

**Scaling limits:** One machine. Roughly 10,000-50,000 tenants depending on activity level and hardware. Approximately 100,000 concurrent subscriptions per node (limited by memory for subscription state and WebSocket connections).

**What this proves:** The core reactive model works. The per-tenant isolation model works. The commit log captures all changes correctly.

**Phase 2: Multi-Node with Tenant Routing (Weeks 9-16)**

**Goal:** Multiple nodes form a cluster, with tenants distributed across nodes and automatic

routing.

**Deliverables:**

- Control plane with tenant → node mapping (embedded Raft via `openraft` or external etcd)
- Node membership and health checking (heartbeat/gossip)
- Application-layer proxy for connection routing
- Tenant migration (file copy + redirect)
- Automatic tenant placement for new tenants (least-loaded node)
- Cluster-wide admin API (list all tenants across all nodes, global schema operations)

**Scaling limits:** 3-50 nodes. Roughly 50,000-500,000 tenants depending on tenant size and activity. Horizontal scaling of total subscription capacity proportional to node count.

**What this proves:** Horizontal scaling works without distributed transactions. Tenant migration is smooth. The system operates as a cluster without single points of failure.

#### Phase 3: Log-Driven Reactive Invalidation (Weeks 17-24)

**Goal:** Replace read-set-based subscription invalidation with commit-log-based invalidation. This is more efficient and enables edge replication.

**Deliverables:**

- Structured commit log entries with per-table, per-index-range change descriptors
- Subscription engine that pattern-matches commit log entries against subscription dependency descriptors
- Index-range-level dependency tracking (e.g., "this subscription depends on messages where channel_id IN (1, 5, 42)")
- Log compaction and truncation with consumer cursor tracking
- Log streaming API for external consumers (gRPC or custom binary protocol over TCP)

**Performance target:** Process 100,000 committed transactions per second through the subscription invalidation engine on a single node, with <5ms from commit to subscription update delivery.

**What this proves:** Reactive queries can scale to high write throughput. The commit log is a viable source for all downstream consumers.

#### Phase 4: Embedded Edge Replicas (Weeks 25-36)

**Goal:** Stream per-tenant commit logs to client-side embedded replicas for zero-latency reactive reads.

**Deliverables:**

- Client SDK with embedded storage engine (redb compiled to the target platform, or SQLite for browser/WASM)
- Log consumer that applies commit log entries to the local replica
- Partial replication — "shapes" or "sync rules" that specify which subset of the tenant's data to replicate (e.g., "only messages in channels the user is a member of")
- Optimistic mutations — the client applies writes locally, sends them to the server, and reconciles when the server confirms or rejects
- Offline write queue — mutations are queued locally when disconnected and replayed on reconnect
- Conflict resolution — last-writer-wins for simple fields, application-defined merge for complex cases
- Client-side reactive query evaluation — subscriptions run against the local replica, with zero network latency for reads

**Consistency model:** The embedded replica provides **read-your-own-writes consistency** (the client always sees its own mutations immediately, even before server confirmation) and **causal consistency** across clients (if Client A's write is confirmed before Client B reads, Client B will see it). This is weaker than the server-side serializable isolation but is the standard for local-first systems.

**What this proves:** The system achieves true local-first performance. Reactive queries are instantaneous. Offline support works.

**Phase 5: Escape Hatches for Large Tenants (Weeks 37+)**

**Goal:** Support the rare tenant that outgrows a single node.

**Deliverables:**

- **Option A: Disaggregated storage for read scaling.** For tenants with high read load, spin up read-only compute nodes that connect to the tenant's storage via a shared storage layer (Neon-style). Subscriptions are distributed across read replicas.
- **Option B: Per-tenant sharding.** For tenants with >100GB of data, partition the tenant's keyspace into ranges across multiple redb files on different nodes. This is shared-nothing sharding but scoped to a single tenant — far simpler than global sharding because cross-shard operations only happen within one tenant's data.
- **Option C: Tiered storage.** Archive old/cold data to object storage (S3) and keep only hot data in redb. Queries against cold data are slower but the active dataset stays small and fast.

**When to implement:** Only when a paying customer needs it. Premature optimization of this layer wastes engineering time — the vast majority of tenants will never exceed a single node's capacity.

---

## 6. Capacity Planning

### 6.1 Per-Node Estimates

Assuming a modern server (32 cores, 128GB RAM, NVMe SSD):

| Metric | Estimate | Bottleneck |
|---|---|---|
| Tenant databases per node | 10,000-50,000 | File descriptors, memory for open databases |
| Active (open) tenant databases | 500-2,000 | RAM for redb buffer pools (~64MB each) |
| Concurrent WebSocket connections | 100,000-500,000 | Memory for connection state (~2KB each) |
| Active subscriptions per node | 100,000-1,000,000 | Memory for dependency tracking (~500B each) |
| Write throughput (mutations/sec) | 50,000-200,000 | Disk I/O (io_uring), CPU for WASM execution |
| Subscription invalidation throughput | 100,000-500,000 events/sec | CPU for pattern matching against subscriptions |
| Commit log streaming throughput | 100MB/sec per consumer | Network I/O |

### 6.2 Cluster Scaling

| Cluster Size | Total Tenants | Total Subscriptions | Total Write Throughput |
|---|---|---|---|
| 1 node | 10,000 | 100,000 | 50,000/sec |
| 5 nodes | 50,000 | 500,000 | 250,000/sec |
| 20 nodes | 200,000 | 2,000,000 | 1,000,000/sec |
| 100 nodes | 1,000,000 | 10,000,000 | 5,000,000/sec |

These are rough estimates. Actual capacity depends heavily on tenant activity distribution (a few hot tenants vs. many cold ones), average query complexity, and subscription fan-out.

### 6.3 Single-Tenant Limits

The maximum size of a single tenant database on one node:

| Resource | Limit | Determined By |
|---|---|---|
| Data size | ~100GB | NVMe capacity, redb file size, working set must fit in node RAM for reactive perf |
| Tables per tenant | ~1,000 | redb overhead per table |
| Rows per table | ~100,000,000 | redb B-tree depth, query performance |
| Subscriptions per tenant | ~50,000 | Memory for dependency tracking, invalidation CPU |
| Writes per second per tenant | ~10,000 | Single-writer serialization in redb |

Tenants exceeding these limits need the Phase 5 escape hatches.

---

## 7. Comparison Matrix

| Property | Range Sharding (CockroachDB) | Disaggregated (Neon) | Database-per-Tenant (This Architecture) |
|---|---|---|---|
| Write scaling | Horizontal (add nodes) | Vertical (single writer) | Horizontal (distribute tenants) |
| Read scaling | Horizontal (followers) | Horizontal (read replicas) | Horizontal (distribute tenants) + edge replicas |
| Reactive query latency | High (cross-range coordination) | Medium (compute→storage hop) | Low (local per-tenant, zero coordination) |
| Distributed transactions | Yes (2PC across ranges) | No (single writer) | No (tenants are isolated) |
| Data isolation | Logical (same DB, RLS) | Logical | Physical (separate files) |

| Property | Range Sharding (CockroachDB) | Disaggregated (Neon) | Database-per-Tenant (This Architecture) |
|---|---|---|---|
| Tenant migration | Range split/merge (complex) | Compute failover (fast) | File copy + redirect (simple) |
| Offline support | No | No | Yes (embedded replicas) |
| Implementation complexity | Very high (~1M LOC) | High (~100K LOC) | Moderate (~50K LOC for core) |
| Single binary deployment | No | No | Yes |
| Cross-tenant queries | Native | Native | Application-level only |

## 8. Risks and Mitigations

### Risk: Hot tenant on a node

**Problem:** One tenant receives disproportionate traffic, overloading its node and degrading other tenants on that node.

**Mitigation:** Per-tenant resource budgets (CPU time, memory, I/O bandwidth) enforced by the tenant manager. If a tenant exceeds its budget, its requests are throttled or queued. For premium tenants, dedicate an entire node. Automatic detection of hot tenants and migration to less-loaded nodes.

### Risk: Schema migration across thousands of databases

**Problem:** A schema change (add column, create index) must be applied to every tenant database.

**Mitigation:** Rolling migrations with version tracking. Each tenant database stores its current schema version. Migrations are applied lazily (when the tenant is next accessed) or eagerly (by a background worker that iterates through all tenants). The system supports running two schema versions simultaneously during the migration window. Schema changes are expressed as deterministic WASM functions, ensuring consistency.

### Risk: Single-tenant size limit

**Problem:** A tenant's data grows beyond what one node can handle.

**Mitigation:** Phase 5 escape hatches (disaggregated storage, per-tenant sharding, tiered storage). More importantly, design the schema and query patterns to keep per-tenant data small. Most

SaaS applications have natural data lifecycle policies (archive old messages, expire sessions, aggregate analytics).

**Risk: Coordination plane failure**

**Problem:** The control plane (tenant → node mapping) becomes unavailable.

**Mitigation:** Data nodes cache the mapping locally. If the control plane is down, existing tenant routing continues to work — only new tenant creation and tenant migration are blocked. The control plane itself is replicated via Raft (3 or 5 nodes) for high availability.

**Risk: Commit log grows unbounded**

**Problem:** If edge replicas fall behind or disconnect, the commit log accumulates entries.

**Mitigation:** Consumer cursor tracking with garbage collection. The log is truncated up to the minimum cursor position across all active consumers. Disconnected consumers that fall too far behind must perform a full resync (snapshot + log replay from the snapshot point) rather than catching up from the log. Configure a maximum log retention period (e.g., 7 days) as a hard limit.

---

## 9. Key Design Principles

1. **The tenant boundary is the scaling boundary.** Never introduce distributed coordination within the tenant hot path. All cross-node coordination (tenant placement, migration, schema registry) is on the control plane, which is off the hot path.
2. **The commit log is the universal integration point.** Every downstream consumer — reactive subscriptions, edge replicas, analytics, backups, audit — reads from the same commit log. This eliminates dual-write problems and ensures all consumers see a consistent, ordered view of changes.
3. **Scale by distributing tenants, not by partitioning data.** Adding capacity means adding nodes and spreading tenants across them. This is operationally simple (file copy, not range split) and preserves all single-node guarantees (serializable isolation, instant reactive invalidation) within each tenant.
4. **Design for the common case, escape-hatch the exceptions.** 99% of tenants fit comfortably on a single node. Optimize the architecture for this case. The 1% that outgrow a node get a different (more complex, more expensive) treatment — but don't let them dictate the architecture for everyone else.
5. **The single binary is sacred.** Every data node runs the same binary with the same code. The control plane can be embedded in the same binary (just a different mode). There is no

separate proxy process, no sidecar, no agent. This is what makes the system easy to deploy: `./reactivedb --mode=data --join=node1:9000,node2:9000,node3:9000`.

---

## 10. Appendix: Prior Art Reference

| System | Architecture Pattern | Key Lesson for This Project |
|---|---|---|
| **CockroachDB** | Range sharding + Raft | How to layer SQL on a KV store. How not to do reactive queries (too much coordination). |
| **FoundationDB** | Unbundled + deterministic log | The commit log as the source of truth for everything. Deterministic simulation testing. |
| **TigerBeetle** | Single-writer + VSR + io_uring | Mechanical sympathy. Static allocation. Deterministic execution enables physical repair across replicas. |
| **Convex** | OCC + reactive subscriptions + V8 | Reactive queries derived from transaction conflict detection. Functions-as-transactions. |
| **Turso/libSQL** | Database-per-tenant + edge replicas | Millions of SQLite databases distributed globally. Embedded replicas for microsecond reads. |
| **Neon** | Disaggregated Postgres | Separation of compute and storage. Copy-on-write branching. Scale-to-zero compute. |
| **Electric SQL** | CRDT sync + Postgres WAL | Shapes API for partial replication. Bi-directional sync with conflict resolution. |
| **Litestream** | WAL streaming to S3 | Continuous backup of embedded databases via log shipping. Simple and effective. |
| **Replicache/Zero** | Client-side mutations + server reconciliation | Optimistic mutations with server authority. The best client-side reactive UX model. |
| **Materialize** | Incremental view maintenance via Differential Dataflow | The theoretical ceiling for reactive query efficiency. Study for Phase 3+ optimization. |